

DeepSeek Sparse Attention

`yuro-py.github.io`

17 August 2026

Abstract

DeepSeek Sparse Attention reduces the computational cost of long-context attention by using a learned Lightning Indexer to rank past tokens and a top- k selector to restrict the subsequent core-attention computation. This note summarizes the Multi-Head Latent Attention foundation, the Lightning Indexer, and the dense warm-up and sparse-training stages.

1 Introduction

DeepSeek published a paper on 2 December 2025 titled *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models* [2]. One of the core breakthroughs described in that work was DeepSeek Sparse Attention, which enables longer context lengths at lower computational cost.

Earlier attention paradigms were dense: for every next token, they processed every preceding token. This creates increasing computational overhead as the context length grows. Sparse attention introduces a Lightning Indexer and a token-selection mechanism before performing core attention.

2 Multi-Head Latent Attention

DeepSeek Sparse Attention is built upon Multi-Head Latent Attention (MLA) [1]. A compact representation of the MLA computation is

$$\begin{aligned}
\mathbf{c}_t^Q &= W^{DQ} \mathbf{h}_t, \\
[\mathbf{q}_{t,1}^C; \mathbf{q}_{t,2}^C; \dots; \mathbf{q}_{t,n_h}^C] &= \mathbf{q}_t^C = W^{UQ} \mathbf{c}_t^Q, \\
[\mathbf{q}_{t,1}^R; \mathbf{q}_{t,2}^R; \dots; \mathbf{q}_{t,n_h}^R] &= \mathbf{q}_t^R = \text{RoPE}(W^{QR} \mathbf{c}_t^Q), \\
\mathbf{q}_{t,i} &= [\mathbf{q}_{t,i}^C; \mathbf{q}_{t,i}^R], \\
\mathbf{c}_t^{KV} &= W^{DKV} \mathbf{h}_t, \\
[\mathbf{k}_{t,1}^C; \mathbf{k}_{t,2}^C; \dots; \mathbf{k}_{t,n_h}^C] &= \mathbf{k}_t^C = W^{UK} \mathbf{c}_t^{KV}, \\
\mathbf{k}_t^R &= \text{RoPE}(W^{KR} \mathbf{h}_t), \\
\mathbf{k}_{t,i} &= [\mathbf{k}_{t,i}^C; \mathbf{k}_{t,i}^R], \\
[\mathbf{v}_{t,1}^C; \mathbf{v}_{t,2}^C; \dots; \mathbf{v}_{t,n_h}^C] &= \mathbf{v}_t^C = W^{UV} \mathbf{c}_t^{KV}, \\
\mathbf{o}_{t,i} &= \sum_{j=1}^t \text{Softmax}_j \left(\frac{\mathbf{q}_{t,i}^\top \mathbf{k}_{j,i}}{\sqrt{d_h + d_h^R}} \right) \mathbf{v}_{j,i}^C, \\
\mathbf{u}_t &= W^O [\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}].
\end{aligned} \tag{1}$$

The main steps are:

1. The query is down-projected to a latent representation, then up-projected into content heads. Rotary positional encoding is applied to the rotary component. The content and rotary components are then concatenated, giving a dimension of “head dimension + rotary-head dimension.”
2. The key and value representations use a down-projected cached latent representation. Keys are up-projected into content heads, while the input is down-projected to a single cached rotary head. The content and rotary components are broadcast and concatenated.
3. Values are up-projected from the down-projected key-value representation.
4. Attention is performed, the heads are concatenated, and the result is passed through a final linear layer.

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \text{ReLU}(\mathbf{q}_{t,j}^I \cdot \mathbf{k}_s^I).$$

Figure 2: Lightning Indexer score.

3 DeepSeek Sparse Attention

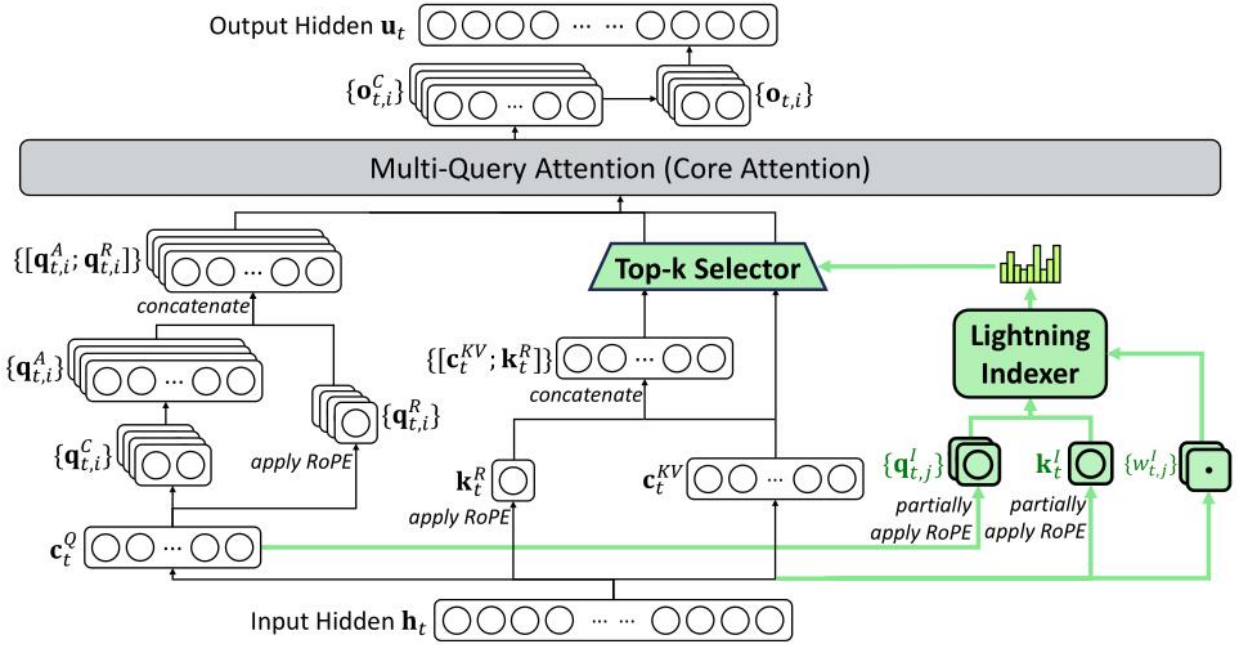


Figure 1: DSA architecture.

During pre-training, dense attention is run before the Lightning Indexer is initialized. In the continual pre-training phase, all parameters except those of the Lightning Indexer are frozen. This allows the indexer to rank and score past-token keys using the preceding dense context.

1. For every token, the query down-projection produces indexer heads. Their number is independent of the content and rotary heads and is typically smaller.
2. The key receives a single indexer head from the input value, which is then broadcast to the query's indexer heads.
3. The Lightning Indexer weights are projected from the input using the same number of heads as the indexer heads.
4. As shown in Figure 2, a dot product between the query and key produces raw scores for past tokens. These scores are passed through the ReLU activation, which removes negative values because they are irrelevant to future computation. The ReLU outputs are weighted individually and summed.

$$\mathbf{u}_t = \text{Attn}(\mathbf{h}_t, \{\mathbf{c}_s \mid I_{t,s} \in \text{Top-k}(I_{t,:})\}).$$

Figure 3: Fine-grained token selector mechanism.

$$\mathcal{L}^I = \sum_t \mathbb{D}_{\text{KL}}(p_{t,:} \parallel \text{Softmax}(I_{t,:})).$$

Figure 4: Dense warm-up stage.

5. The Lightning Indexer output is passed to the top- k function, shown in Figure 3. Depending on the hyperparameter k , this function retrieves the indices of the top- k ranked keys. The paper uses $k = 2048$. This is the “sparse” component of the method.
6. The indices of the selected keys are used to retrieve the corresponding key-value pairs for core attention.

4 Training Stages

4.1 Dense Warm-Up Stage

In this pre-training phase, the Lightning Indexer is warmed up and initialized. All parameters except those of the Lightning Indexer are frozen, and there is no token-selection mechanism. This forces the indexer to learn the patterns of dense attention.

The indexer outputs and main-attention outputs must be aligned for the current token. The outputs are summed across all attention heads and then L1-normalized along the sequence dimension to produce a target distribution. The indexer output is passed through a softmax to produce a predicted distribution, which is compared with the target distribution to obtain the KL-divergence loss.

- Learning rate: 10^{-3}
- Training steps: 1000
- Sequences: 16
- Tokens per sequence: 128000
- Total tokens: 2.1 B

4.2 Sparse Training Stage

In the continued pre-training phase, all parameters are unfrozen and the Lightning Indexer’s output is filtered by a token-selection mechanism. The entire model is optimized to adapt to sparsity.

The KL-divergence loss continues to align the indexer outputs with the main attention outputs, but it now depends only on the selected tokens. The indexer head is detached from the computational

graph and continues to receive the KL-divergence training signal, while the main model receives only the language-modeling loss.

- Learning rate: 7.3×10^{-6} (lower)
- Training steps: 15000 (higher)
- Sequences: 480 (higher)
- Tokens per sequence: 128000 (same)
- Total tokens: 943.7 B

The sparse-stage loss is

$$\mathcal{L}^I = \sum_t \mathbb{D}_{\text{KL}}(p_{t,S_t} \parallel \text{Softmax}(I_{t,S_t})). \quad (2)$$

Here, S_t denotes the set of tokens selected for position t .

References

- [1] DeepSeek-AI et al., “DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model,” *arXiv preprint arXiv:2405.04434*, 2024. Available at: <https://arxiv.org/pdf/2405.04434>.
- [2] DeepSeek-AI et al., “DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models,” *arXiv preprint arXiv:2512.02556*, 2025. Available at: <https://arxiv.org/pdf/2512.02556>.